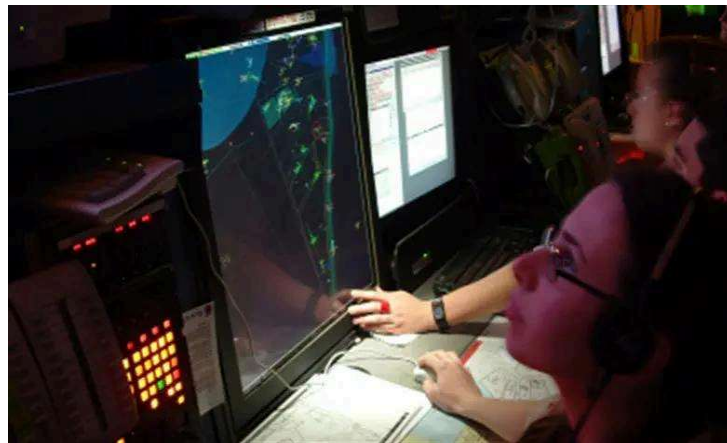


# Iron Dome Ops

## מערכת ניהול אירועים מבצעיים



---

## סיפור רקע

יחידת הגנה אווירית מפעילה מערכת פנימית לניהול אירועים מבצעיים בזמן אמת.

במהלך פעילות מבצעית מתקבלים דיווחים ממערכות מכ"ם וחיישנים שונים.  
כאשר מזוהה איום — נפתח Incident מבצעי במערכת.

המפעילים יכולים:

- לפתוח אירוע
  - לעדכן סטטוס אירוע
  - לצפות באירועים פתוחים
  - לתעד פעולות שבוצעו
  - המערכת תשמור את כל הנתונים באמצעות: MySQL
- 

## מטרת הפרויקט

לבנות שרת Backend באמצעות:

- Node.js
- Express.js
- MySQL
- mysql2
- Docker

תוך שימוש בארכיטקטורה בסיסית ומבנה תיקיות מסודר.

---

## דרישות הפרויקט

### 1. יצירת Operator

Endpoint

POST /operators

Body

```
{  
  "name": "John",  
  "rank": "Captain"  
}
```

---

## 2. פתיחת Incident מבצעי

Endpoint

POST /incidents

Body

```
{  
  "codeName": "RED SKY",  
  "threatLevel": "HIGH",  
  "operatorId": 1  
}
```

---

## 3. שינוי סטטוס Incident

Endpoint

PATCH /incidents/:id/status

Body

```
{  
  "status": "INTERCEPTED"  
}
```

סטטוסים אפשריים:

- OPEN
  - TRACKING
  - INTERCEPTED
  - CLOSED
- 

## 4. שליפת כל האירועים הפתוחים

GET /incidents/open

---

## דרישות בסיס הנתונים

יש להשתמש ב-MYSQL.

### Table: operators

| Column | Type                           |
|--------|--------------------------------|
| id     | INT AUTO_INCREMENT PRIMARY KEY |
| name   | VARCHAR(100)                   |
| rank   | VARCHAR(100)                   |

---

### Table: incidents

| Column    | Type                           |
|-----------|--------------------------------|
| id        | INT AUTO_INCREMENT PRIMARY KEY |
| code_name | VARCHAR(100)                   |

|              |                                     |
|--------------|-------------------------------------|
| threat_level | VARCHAR(50)                         |
| status       | VARCHAR(50)                         |
| operator_id  | INT FK(operators(id))               |
| created_at   | TIMESTAMP DEFAULT CURRENT_TIMESTAMP |

---

**Table: logs**

| Column      | Type                                |
|-------------|-------------------------------------|
| id          | INT AUTO_INCREMENT PRIMARY KEY      |
| action      | VARCHAR(100)                        |
| incident_id | INT FK(incidents(id))               |
| operator_id | INT FK(operators(id))               |
| description | TEXT                                |
| created_at  | TIMESTAMP DEFAULT CURRENT_TIMESTAMP |

---

# Foreign Keys

- incidents.operator\_id → operators.id
  - logs.operator\_id → operators.id
  - logs.incident\_id → incidents.id
- 

## דוגמאות ל-Logs

בעת יצירת Incident

| action           | description          |
|------------------|----------------------|
| INCIDENT_CREATED | New incident created |

---

בעת שינוי סטטוס

| action         | description                   |
|----------------|-------------------------------|
| STATUS_UPDATED | Status changed to INTERCEPTED |

---

## ארכיטקטורה מומלצת

- project/
- |
- |— controllers/
- |— services/
- |— repositories/
- |— routes/

- db/
  - middleware/
  - utils/
  - app.js
  - Dockerfile
  - docker-compose.yml
  - readme.md
  - .env
- על קובץ זה לכלול הוראות ברורות להרצת הפרוייקט - לדוגמא, איזו 'משתני סביבה הכרחיים וכד, DOCKER, פקודה תריך לי את ה

---

## חלוקת אחריות

### Routes

מגדירים Endpoints בלבד.

---

### Controllers

מקבלים

- request
- response

ואינם מכילים לוגיקה עסקית.

---

### Services

אחראים על הלוגיקה העסקית.

לדוגמה:

- יצירת Incident
- שינוי Status
- יצירת Log
- בדיקות תקינות

---

## Repositories

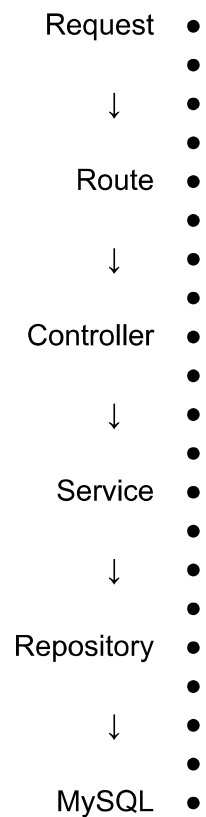
אחראים על כל העבודה מול MySQL.

אין לבצע שאילתות SQL מתוך ה-Service.

כל שאילתות ה-SQL יהיו אך ורק בתוך שכבת Repository.

---

## Flow העבודה



---

## דרישות טכניות

יש להשתמש ב:



- Express •
  - mysql2 •
  - dotenv •
  - Docker •
  - Docker Compose •
- 

## Dependencies

```
npm install express mysql2 dotenv
```

---

## Dev Dependency

```
npm install --save-dev nodemon
```

---

## Docker

יש ליצור Container של MySQL באמצעות Docker Compose.

דוגמה:

services:

mysql:

image: mysql:8

container\_name: iron-dome-db

restart: always

ports:

- "3306:3306"

environment:

MYSQL\_ROOT\_PASSWORD: root

MYSQL\_DATABASE: iron\_dome

volumes:

- mysql-data:/var/lib/mysql

volumes:

mysql-data:

---

## קובץ .env

- PORT=3000
- 
- DB\_HOST=localhost
- DB\_PORT=3306
- DB\_USER=root
- DB\_PASSWORD=root
- DB\_NAME=iron\_dome

---

## חיבור למסד

יש ליצור קובץ

db/database.js

לדוגמה:

```
import mysql from "mysql2/promise";

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});

export default pool;
```

---

## יצירת בסיס הנתונים

לאחר עליית ה־Container יש ליצור את הטבלאות באמצעות קובץ SQL.

לדוגמה:

database.sql

שיכול:

- CREATE DATABASE
- CREATE TABLE operators
- CREATE TABLE incidents
- CREATE TABLE logs
- Foreign Keys

---

## דגשים חשובים

### 1. הפרדת אחריות

אין לכתוב את כל הקוד בקובץ אחד.

---

### 2. קוד מסודר

שמות ברורים לפונקציות, משתנים וקבצים.

---

### 3. Error Handling

יש לטפל לפחות במקרים הבאים:

- Incident לא קיים
- Operator לא קיים
- Status לא חוקי
- שגיאת התחברות למסד הנתונים

---

## בונוס (לא חובה)

- Validation
- Middleware לטיפול בשגיאות
- שימוש ב-Foreign Keys
- שימוש ב-DEFAULT CURRENT\_TIMESTAMP
- שימוש ב-Connection Pool של mysql2

---

## זמן עבודה מומלץ

זמן

שלב

30 דקות

Setup של Docker והרצת MySQL

30 דקות

יצירת בסיס הנתונים והטבלאות

שעה ורבע

Repository + Services

שעה

Routes + Controllers

שעה

ארכיטקטורה וניקוי קוד

---

## מה תלמדו מהפרויקט?

- עבודה עם REST API
- עבודה מול MySQL באמצעות mysql2
- עבודה עם Docker ו-Docker Compose
- עבודה עם מספר טבלאות וקשרים (Relations)
- ארכיטקטורת Backend בסיסית
- הפרדת אחריות בין שכבות
- עבודה עם Express
- תכנון מערכת בצורה מקצועית

---

## דוגמאות ל-Threat Levels

- LOW
- MEDIUM
- HIGH

## דוגמאות ל-Code Names

- RED SKY
  - BLACK FALCON
  - IRON SHIELD
  - NIGHT ARROW
  - SILENT DOME
- 

## דרישות Docker

לפני תחילת פיתוח השרת יש לבצע את השלבים הבאים:

1. ליצור קובץ `.docker-compose.yml`
2. להרים Container של MySQL באמצעות:

```
docker compose up -d
```

3. לוודא שה-Container רץ באמצעות:

```
docker ps
```

4. להתחבר לשרת MySQL באמצעות כלי כלשהו או שורת הפקודה.
5. ליצור את בסיס הנתונים והטבלאות באמצעות קובץ `.database.sql`.
6. רק לאחר מכן להתחיל בפיתוח ה-Backend ב-Node.js תוך שימוש ב-`mysql2`.